

Introduction

University life includes not only the educational process but also living conditions, which directly affect students' academic performance, discipline, and overall quality of life. One of the key elements of university infrastructure is student dormitories, which provide accommodation for students—especially those coming from other cities—and create an environment for daily life and preparation for studies. Therefore, organizing and managing dormitory operations is an important task that requires accurate record-keeping, efficient management, and control at multiple levels.

Dormitory management is a complex set of processes that includes student registration, allocation of students to rooms and floors, monitoring room availability, updating information during relocations, and providing up-to-date data for university administration. In practice, these processes are often carried out manually or using fragmented tools, which leads to a number of problems: information becomes outdated quickly, the risk of data duplication increases, it becomes difficult to track changes, and report generation takes a significant amount of time. Moreover, as the number of students, rooms, and dormitories grows, the workload on staff increases, reducing operational efficiency and causing additional organizational difficulties.

An additional challenge arises from the multi-level management structure. On the one hand, university administration must be able to oversee multiple dormitories and obtain a general view of occupancy rates, room distribution, and infrastructure status. On the other hand, the administration of a specific dormitory must manage internal details such as floors, rooms, resident lists, relocations, and the current occupancy status. In the absence of a unified digital system, errors and inconsistencies occur, which negatively affect both staff performance and student convenience.

For these reasons, the development of a modern information system becomes highly relevant. Such a system allows data centralization, automation of key operations, and improved transparency of processes. This approach helps reduce the time required for routine tasks, increase the accuracy of records, and provide a convenient tool for all participants involved. An important requirement is the implementation of access control, since different categories of users must have different levels of permissions and access to data.

This course project is devoted to the development of an online student dormitory management system that includes a mobile application and a web-based administration panel (Admin Panel). The mobile application provides access to the system from an Android device and enables convenient execution of operations at any time. The server side implements business logic, request processing, access control, and secure data storage. The web panel is designed for administrative tasks and serves as an additional interface that allows administrators to efficiently manage dormitories through a browser.

The developed system supports three user levels that correspond to the real administrative structure:

- **University Administrator (University Admin)** — the top management level with access to information about all university dormitories. This role allows creating dormitories, editing their parameters, assigning dormitory administrators (Dorm Admins), and linking them to specific buildings. This ensures centralized control and structured management.
- **Dormitory Administrator (Dorm Admin)** — the operational level responsible for managing a specific dormitory. This role includes managing floors and rooms, adding and editing student data, and performing room assignment operations. During assignment, the system must take room capacity into account in order to prevent over-occupancy and ensure correct allocation.
- **Student (Student)** — a user with limited permissions whose main task is to obtain information about their accommodation. A student can log in through the mobile application and view details of their room and floor, as well as information about roommates. This increases transparency and convenience, allowing students to access necessary information without contacting administration.

The technologies used in this project were selected to meet the requirements of mobility, reliability, and scalability. **Flutter** and the **Dart** programming language are used for the mobile client, as they enable rapid development of modern user interfaces and provide stable performance on Android devices. The server side is built using **Laravel**, which offers a well-structured architecture for REST API development, built-in security mechanisms, and authentication support. **MySQL** is used as the database management system, since the relational data model is well suited for storing interconnected entities such as dormitories, floors, rooms, students, and room assignments.

Thus, the developed system represents a practical solution aimed at improving the efficiency of dormitory management and enhancing interaction between administration and students. The project also has significant potential for further expansion, such as adding notifications, maintenance requests, reports, and analytical tools, which makes it suitable for real-world application in a university environment.

Chapter 1. Theoretical Part

1.1 Relevance of the Topic

The automation of student dormitory management is a highly relevant topic in the context of modern higher education institutions. Universities today aim to digitalize administrative processes in order to increase efficiency, reduce manual work, and improve the quality of services provided to students. Dormitories are complex organizational units that involve constant interaction between administration and residents, which makes manual management inefficient and error-prone.

The use of mobile applications and web-based administration panels allows universities to significantly speed up administrative operations, ensure data accuracy, and provide real-time access to information. Digital systems also improve transparency and communication between different management levels and students, which directly enhances the overall student experience. Therefore, developing an automated dormitory management system is both a practical and academically significant task.

1.2 Problem Description

Traditional dormitory management is associated with several critical problems that negatively affect operational efficiency. These problems include:

- Lack of a unified and centralized database for students, rooms, and dormitories;
- Difficulty in controlling room occupancy and preventing capacity violations;
- Time-consuming search for information and report generation;
- High risk of errors during student relocation or reassignment;
- Limited transparency for students regarding their room details, roommates, and dormitory information;
- Inefficient communication between different administrative levels.

As the number of students and dormitories increases, these issues become more severe, highlighting the necessity for a structured digital solution.

1.3 Purpose and Objectives of the Project

Project Purpose

The main purpose of this project is to develop an online student dormitory management system that automates administrative processes and provides role-based access for different categories of users.

Project Objectives

To achieve this goal, the following objectives were defined:

1. To design and implement a secure authentication system using email and password;
2. To implement a role-based access control system with multiple administrative levels;
3. To develop CRUD (Create, Read, Update, Delete) modules for dormitories, floors, rooms, and students;
4. To implement a student room assignment mechanism with capacity validation;
5. To develop a REST API to ensure interaction between the mobile application and the server;
6. To create a mobile application using Flutter for administrators and students;
7. To develop a web-based administration panel for advanced management;
8. To implement reporting, announcements, and ticketing functionalities;
9. To provide data import and export capabilities;
10. To prepare documentation and instructions for system deployment and usage.

1.4 Object and Subject of the Study

The **object of the study** is the dormitory management process, including student registration, room assignment, dormitory administration, and data control.

The **subject of the study** includes software development methods, architectural approaches, and technologies used in building mobile and web-based information systems, specifically using Flutter, Laravel, and MySQL.

1.5 Analysis of Existing Solutions

Existing dormitory management solutions are typically presented in one of the following forms:

- Web-based systems without mobile support;
- Mobile applications with limited administrative functionality;
- Commercial enterprise systems that require high costs and complex integration.

Many available systems lack advanced role separation, real-time data synchronization, or flexibility for customization. Additionally, paid solutions are often unsuitable for educational purposes. Therefore, this course project proposes a scalable academic solution that can be extended and adapted for real university environments.

1.6 Proposed Solution

The proposed solution is a comprehensive online dormitory management system that consists of:

- A **Flutter-based mobile application** for administrators and students;
- A **Laravel-based server side (REST API)** responsible for security, business logic, and data processing;
- A **MySQL database** for reliable storage of structured and relational data;
- A **web-based Admin Panel** for advanced administrative operations.

1.7 System Roles and Permissions

The system implements a structured role-based access model with four main roles:

Super Administrator (Super Admin)

The Super Admin is the highest authority in the system and is responsible for global configuration and system-wide management. This role includes:

- Managing system notifications;
- Performing global search across students, rooms, floors, and dormitories;
- Accessing all system data without restrictions;
- Managing system settings and themes;
- Monitoring activity and system-wide reports.

University Administrator (University Admin)

The University Admin manages all dormitories belonging to the university and has the following capabilities:

- Creating and managing dormitories;
- Adding and managing dormitory administrators;
- Publishing announcements for dormitories and students;
- Generating and viewing reports;
- Managing and responding to support tickets;
- Accessing dashboards and statistical summaries;
- Managing accounts, students, dorms, and dorm administrators;
- Importing and exporting data (CSV/Excel/PDF).

Dormitory Administrator (Dorm Admin)

The Dorm Admin is responsible for daily operations within a specific dormitory. This role allows:

- Managing student records and registrations;
- Managing floors and rooms;
- Assigning and reassigning students to rooms;
- Viewing unassigned students and assignment history;
- Managing dormitory announcements;
- Handling support tickets and responding to student requests;
- Generating dormitory-level reports;
- Archiving, restoring, freezing, and unfreezing records.

Student

The Student role provides access to personal accommodation information and basic interactions:

- Viewing personal room and dormitory information;
- Viewing roommates and room details;
- Submitting support tickets or requests;
- Receiving announcements and notifications;
- Accessing personal account information.

1.8 Functional Features of the System

The developed system includes the following key features:

- Email and password authentication;
- Role-based dashboards;
- Student assignment and reassignment (move) between rooms;
- Viewing room details and occupancy status;
- Assignment history tracking;
- Floor and room management;
- Import and export of data;
- Advanced search and filtering;
- Creation, editing, deletion, archiving, restoring, freezing, and unfreezing of records;
- Notification system;
- Announcements module;
- Reports and analytics;
- Secure logout and session control.

1.9 Summary of the Theoretical Part

The theoretical analysis confirms that traditional dormitory management approaches are inefficient in modern educational environments. The proposed system offers a structured, scalable, and role-based solution that addresses existing challenges through automation, transparency, and digital transformation. The use of modern technologies ensures reliability, security, and the possibility of further system development.

Chapter 2. Practical Part

2.1 System Requirements

2.1.1 Functional Requirements

The developed system must provide a complete set of functional requirements to support dormitory management operations at different administrative levels. These requirements include:

- User registration and authentication using email and password;
- Secure login with token-based authentication;
- Management of dormitories and dormitory administrators by the **University Administrator**;
- Management of floors, rooms, and students by the **Dormitory Administrator**;
- Assignment of students to rooms with automatic capacity validation;
- Support for student relocation and reassignment between rooms;
- Viewing room details, occupancy status, and assignment history;
- Announcement management for administrators;
- Ticket and support request submission and response handling;
- Report generation and viewing for administrators;
- Data import and export (CSV, Excel, PDF);
- Advanced search and filtering across students, rooms, floors, and dormitories;
- Student access to personal accommodation information, including room details and roommates.

2.1.2 Non-Functional Requirements

In addition to functional requirements, the system must satisfy the following non-functional requirements:

- **Security:** token-based authentication using Laravel Sanctum and role-based authorization;
- **Data Integrity:** reliable storage and processing of data using MySQL;
- **Performance:** fast response time for API requests and UI interactions;
- **Usability:** intuitive user interface for mobile (Flutter UI) and web (Admin Panel);
- **Scalability:** ability to add new modules such as maintenance requests, notifications, and analytics;
- **Maintainability:** modular architecture with clear separation of concerns;
- **Reliability:** consistent system behavior under concurrent access.

2.2 Project Architecture (Client–Server Model)

The system is built using a client–server architecture, which ensures flexibility, scalability, and centralized data management.

Architecture structure:

Flutter Mobile Application → Laravel REST API → MySQL Database

- The **Flutter mobile application** acts as the client side and provides user interaction through a modern and responsive interface.
- The **Laravel REST API** is responsible for request handling, business logic, role validation, capacity checks, and security enforcement.
- The **MySQL database** stores all structured and relational data related to dormitories, rooms, students, and assignments.

This architecture allows the system to support multiple clients and ensures that business rules are enforced consistently at the server level.

2.3 Database Design (ERD)

The database is designed using a relational model to represent real-world entities and their relationships accurately.

Main Entities

- **Users** — system user accounts with authentication credentials and roles;
- **Dorms** — university dormitory buildings;
- **DormAdmins** — association between dormitory administrators and assigned dormitories;
- **Floors** — floors within a dormitory;
- **Rooms** — rooms with defined capacity and status;
- **Students** — student profiles linked to user accounts;
- **RoomAssignments** — records of student room assignments, including history and active assignment.

The **RoomAssignments** entity enforces the rule that each student can have only one active room assignment at a time, ensuring data consistency and preventing conflicts.

2.4 API Design

The system exposes a RESTful API that enables communication between the mobile application, the web admin panel, and the backend.

Authentication Endpoints

- POST /api/login — user authentication and token generation;
- GET /api/me — retrieve authenticated user profile;
- POST /api/logout — invalidate authentication token.

University Administrator Endpoints

- CRUD operations for dormitories:
 - GET /api/dorms
 - POST /api/dorms
 - PUT /api/dorms/{id}
 - DELETE /api/dorms/{id}
- Create and assign dormitory administrators:
 - POST /api/dorms/{id}/create-dorm-admin
- Announcement publishing and report access.

Dormitory Administrator Endpoints

- CRUD operations for floors:
 - GET /api/floors
 - POST /api/floors
 - PUT /api/floors/{id}
 - DELETE /api/floors/{id}
- CRUD operations for rooms:
 - GET /api/rooms
 - POST /api/rooms
 - PUT /api/rooms/{id}
 - DELETE /api/rooms/{id}
- CRUD operations for students:
 - GET /api/students
 - POST /api/students
 - PUT /api/students/{id}
 - DELETE /api/students/{id}
- Room assignment operations:
 - POST /api/rooms/{roomId}/assign-student
 - GET /api/rooms/{roomId}/occupants
- Ticket management and reporting.

Student Endpoints

- GET /api/student/my-room — retrieve student room information and roommates;
- Ticket submission and announcement viewing.

2.5 Flutter Mobile Application Implementation

The mobile application is developed using **Flutter** and the **Dart** programming language. Flutter was selected for its ability to create high-performance mobile applications with a single codebase.

Technologies Used

- **Dio** — for HTTP communication with the REST API;
- **go_router** — for application routing and navigation;
- **SharedPreferences** — for secure local storage of authentication tokens.

Main Screens

- Login Screen;
- University Admin Dashboard;
- Dorm Admin Dashboard (Floors, Rooms, Students);
- Room Details and Student Assignment Screen;
- Student Home Screen (My Room);
- Tickets, Announcements, and Reports Screens.

2.6 Laravel Backend Implementation

The backend is implemented using **Laravel**, which provides a structured framework for building secure and scalable APIs.

Key Backend Components

- Database migrations and seeders for schema and test data;
- REST API controllers with **FormRequest** validation;
- Middleware for role-based access control;
- Business logic services for room assignment and relocation;
- Reporting, ticketing, and notification modules;
- Import/export and search functionality.

Laravel Sanctum is used to provide secure token-based authentication for mobile clients.

2.7 Role-Based Access Control and Permissions

The system implements strict role-based access control to ensure data security and proper responsibility separation:

- **Super Admin** — system-wide access, notifications management, global search, system settings, and monitoring;
- **University Admin** — full control over dormitories, dorm administrators, announcements, reports, and tickets;
- **Dorm Admin** — access limited to a specific dormitory, including students, rooms, floors, assignments, reports, and announcements;
- **Student** — access limited to personal accommodation data and ticket submission.

This approach significantly improves system security, prevents data leakage, and ensures controlled access to sensitive information.

2.8 Practical Part Summary

The practical implementation demonstrates how modern technologies can be combined to build a reliable and scalable dormitory management system. By applying a client–server architecture, role-based access control, and modular design, the system effectively addresses real-world administrative challenges and provides a solid foundation for further development and enhancement.

Chapter 3. Planning and Workflow Design

3.1 Project Plan and Development Stages

The development of the dormitory management system was organized into clearly defined stages in order to ensure structured implementation, reduce risks, and maintain code quality. Dividing the project into stages made it possible to validate functionality at each step and gradually integrate all system components.

The project was implemented according to the following stages:

1. **Database and Role Preparation**

At this stage, the database schema was designed and implemented using MySQL. Core entities such as users, dormitories, floors, rooms, students, and assignments were created. User roles and access permissions were defined to support role-based access control.

2. **API and CRUD Module Development**

The second stage focused on developing the REST API using Laravel. CRUD (Create, Read, Update, Delete) operations were implemented for all core entities. Business rules, such as room capacity validation and role-based access restrictions, were integrated into the server-side logic.

3. **Flutter Mobile Application Implementation**

During this stage, the mobile application was developed using Flutter and Dart. The application was connected to the REST API, authentication logic was implemented, and user interfaces were created according to user roles (administrators and students).

4. **Testing and Documentation Preparation**

The final stage included functional testing of all modules, validation of business logic, and fixing detected issues. Technical documentation, system usage instructions, and project description were prepared to support deployment and evaluation.

3.2 UML / Use Case Analysis

Use Case analysis was applied to describe the interaction between users and the system. Each user role has a defined set of actions that reflect real operational scenarios.

University Administrator (University Admin)

- Manage dormitories (create, edit, delete);
- Create and assign dormitory administrators;
- View system dashboards, announcements, and reports.

Dormitory Administrator (Dorm Admin)

- Manage floors within a dormitory;
- Manage rooms and room parameters;

- Manage student records;
- Assign students to rooms;
- View assignment history and occupancy status;
- Handle tickets, announcements, and reports related to the dormitory.

Student

- View personal room and dormitory information;
- View roommate details;
- Submit tickets or requests;
- Read announcements and notifications.

This role-based Use Case model ensures clarity in responsibilities and simplifies system navigation.

3.3 Sequence Diagram Description

The sequence diagram illustrates the interaction between system components during key operations. One of the most important processes is student room assignment.

Sequence of Student Assignment

1. The **Dorm Admin** selects a student and a target room in the system interface;
2. The **Flutter mobile application** sends a request to the Laravel REST API;
3. The **Laravel backend** validates the request and checks room capacity and assignment rules;
4. If validation is successful, a new **RoomAssignment** record is created and the room status is updated;
5. The API returns a success response to the mobile application;
6. The **Flutter application** updates the user interface to reflect the new assignment.

This sequence ensures data integrity and prevents room over-occupancy.

3.4 User Interface Flow (UI Flow)

The UI Flow defines navigation between application screens based on user roles and access permissions. After application launch, the system determines the user role and displays the appropriate interface.

UI Navigation Flow

- **Splash Screen** → Authentication check;

- **Login Screen** → User authentication via email and password;

After successful login:

- **University Admin Flow:**
University Admin Home → Dormitories → Create Dorm Admin → Reports / Announcements / Dashboard
- **Dorm Admin Flow:**
Dorm Admin Home → Floors / Rooms / Students → Assign Student → Tickets / Reports / Announcements
- **Student Flow:**
Student Home → My Room → Roommates / Dorm Information → Tickets / Notifications

This structured UI flow improves usability, reduces learning time, and ensures a clear user experience for all roles.

3.5 Chapter Summary

This chapter presented the planning approach and workflow design of the dormitory management system. The structured development stages, clearly defined use cases, well-defined sequence interactions, and logical user interface flow ensure that the system is easy to use, scalable, and suitable for real-world university environments.

Chapter 4. Technologies Used

The development of the student dormitory management system required the selection of technologies that ensure reliability, scalability, and ease of maintenance. Each technology was chosen based on its suitability for the project requirements and its ability to support future system expansion.

4.1 Flutter and Dart

Flutter is a modern cross-platform framework used for developing mobile applications with a single codebase. It allows developers to build high-performance applications with a responsive and visually appealing user interface. The use of Flutter significantly reduces development time and ensures consistency across different devices.

The Dart programming language was used together with Flutter, as it is optimized for UI development and provides features such as strong typing, asynchronous programming, and high execution performance. This combination makes Flutter and Dart well suited for developing a mobile client that supports multiple user roles.

4.2 Laravel Framework

Laravel was selected as the backend framework due to its robust architecture and support for REST API development. It provides built-in tools for routing, validation, authentication, and authorization, which simplifies the implementation of business logic and security mechanisms.

The Laravel framework supports a clean project structure, making the codebase easier to maintain and extend. It also enables the implementation of role-based access control, reporting modules, and integration with external services.

4.3 MySQL Database

MySQL was chosen as the database management system because it is a reliable and widely used relational database. It is well suited for storing structured data with clear relationships, such as dormitories, floors, rooms, students, and room assignments.

The relational model allows enforcing data integrity through foreign keys and constraints, ensuring consistency and preventing data anomalies.

4.4 Laravel Sanctum

Laravel Sanctum is used to provide token-based authentication for mobile clients. It allows secure user authentication and ensures that only authorized users can access protected API endpoints.

Sanctum simplifies the authentication process for mobile applications and integrates seamlessly with Laravel, making it an appropriate choice for this project.

4.5 Blade and CSS (Bootstrap)

The web-based administration panel is implemented using Laravel Blade templates combined with CSS and the Bootstrap framework. Blade provides a clean and structured way to build dynamic web pages, while Bootstrap ensures responsive design and a consistent visual layout.

This combination allows administrators to manage dormitories, students, rooms, reports, and system settings efficiently through a browser-based interface.

Chapter 5. Implementation Plan

To ensure systematic development and timely completion of the project, the implementation process was divided into several development sprints. Each sprint focuses on a specific set of tasks and deliverables.

Sprint 1

- Configuration of Laravel backend and MySQL database;
- Implementation of user authentication and role-based access control;
- Development of CRUD functionality for dormitories.

Sprint 2

- Development of CRUD modules for floors and rooms;
- Implementation of room occupancy status calculation and capacity validation.

Sprint 3

- Development of CRUD functionality for students;
- Implementation of student room assignment logic;
- Development of Flutter user interfaces for all system roles.

Sprint 4 (Optional)

- User interface improvements and design enhancements;
- Implementation of additional reporting and analytical features.

Chapter Summary

The selected technologies and structured implementation plan ensure that the system is reliable, scalable, and maintainable. The sprint-based development approach allows incremental delivery of functionality and provides flexibility for future system expansion.

Chapter 6. System Deployment and Operation Guide

This chapter provides detailed instructions for deploying and running the developed dormitory management system. The deployment process includes setting up the database, launching the backend API, and running the mobile application.

6.1 MySQL Database Setup

The first step in deploying the system is preparing the database environment. The system uses MySQL as the primary database management system.

The following steps must be performed:

1. Install **MySQL Server** on the local machine or run it using **Docker**;
2. Create a new database with the name `dorm_management`;
3. Configure the database connection parameters in the Laravel environment file `.env`, including:
 - Database host;
 - Database name;
 - Database username;
 - Database password.

Correct database configuration is required for successful communication between the backend and the database.

6.2 Running the Laravel REST API

After configuring the database, the backend server can be launched. The backend is implemented using the Laravel framework and provides a REST API for the mobile application and admin panel.

The following commands should be executed in the backend project directory:

```
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan serve
```

- `composer install` installs all required backend dependencies;
- `cp .env.example .env` creates the environment configuration file;
- `php artisan key:generate` generates the application encryption key;
- `php artisan migrate --seed` creates database tables and inserts test data;

- `php artisan serve` starts the Laravel development server.

After successful execution, the API will be available at:

`http://127.0.0.1:8000`

6.3 Running the Flutter Mobile Application

The mobile application is developed using Flutter and must be launched separately.

The following steps should be performed in the Flutter project directory:

```
flutter pub get  
flutter run
```

- `flutter pub get` installs all required Flutter dependencies;
- `flutter run` builds and launches the application on a connected Android device or emulator.

Important Note

When using an **Android Emulator**, the API base URL must be set to:

`http://10.0.2.2:8000`

This address allows the emulator to access the local backend server.

Chapter Summary

Following the steps described above ensures correct deployment and operation of the dormitory management system. The system is designed to be easy to install and run, allowing administrators and students to start using the application with minimal configuration.

Chapter 7. Project Screenshots

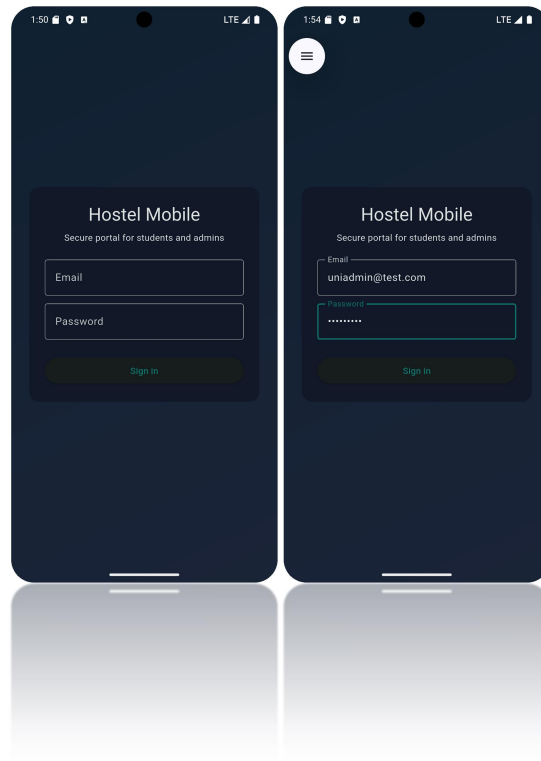
This chapter presents screenshots of the main application screens in order to visually demonstrate the implemented functionality and user interface.

The following key screens are included in this section:

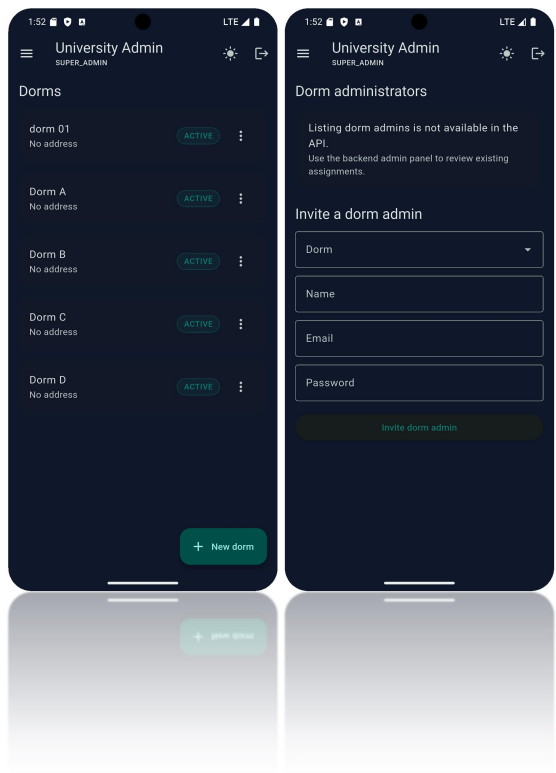
- **Login Screen** — user authentication using email and password;
- **Dormitory List Screen (University Admin)** — management of university dormitories;
- **Floors and Rooms Management Screen (Dorm Admin)** — management of floors, rooms, and room capacity;
- **Students List and Room Assignment Screen** — registration of students and assignment to rooms;
- **Student Screen: My Room** — display of personal room information and roommates.

Example of Screenshot Presentation

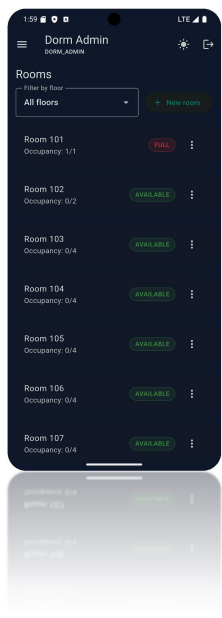
- *Figure 1 — Login Screen*



• *Figure 2 — Dormitory List Screen*



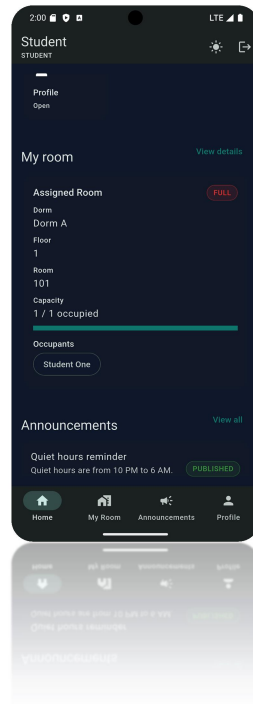
• *Figure 3 — Floors and Rooms Management Screen*



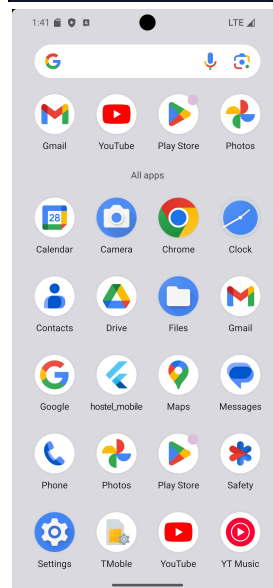
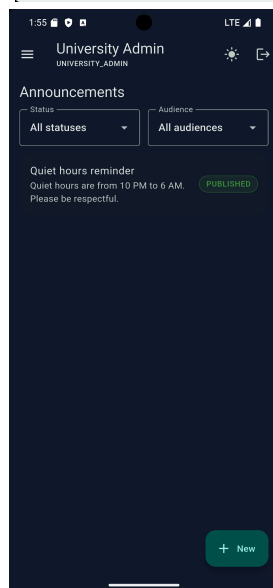
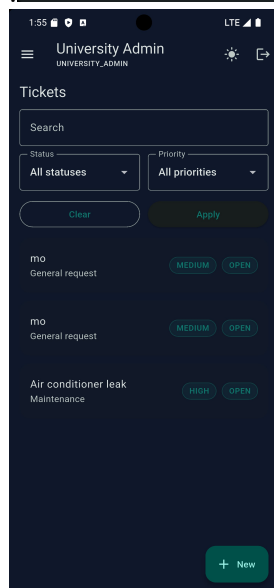
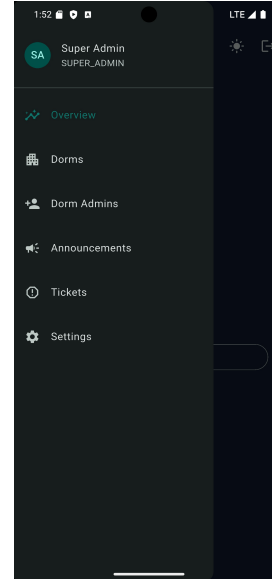
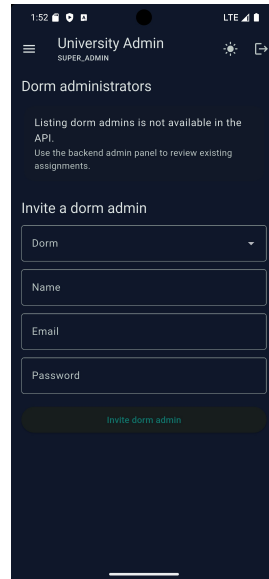
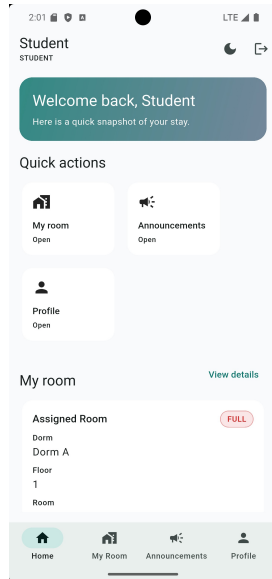
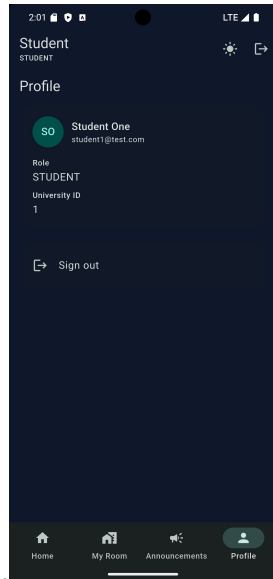
- *Figure 4 — Students List and Room Assignment*

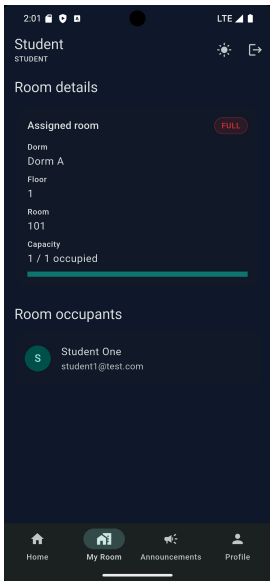
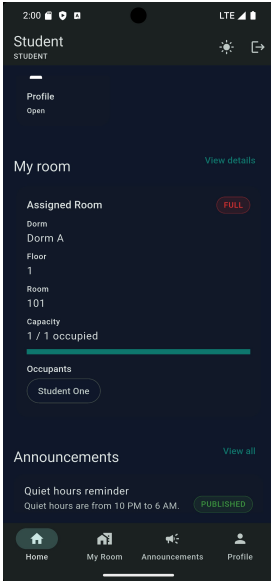
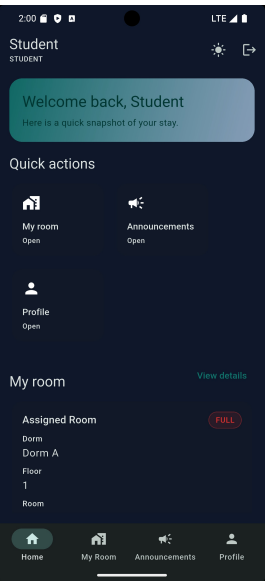
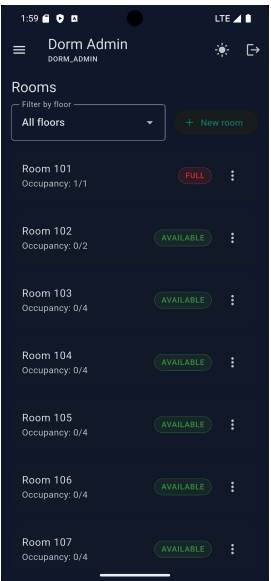
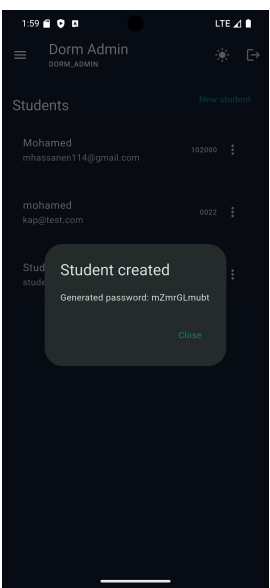
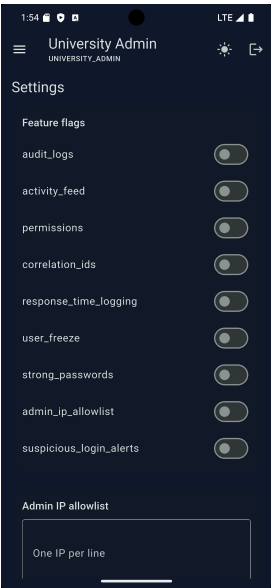
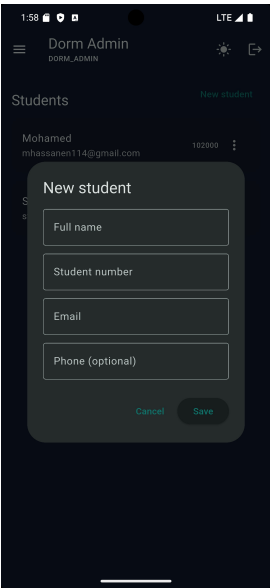
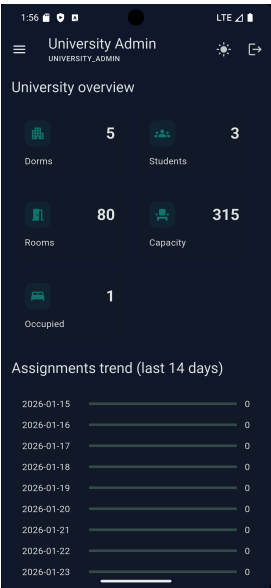


- *Figure 5 — Student “My Room” Screen*









Chapter 8. Future Development

The developed system provides a solid foundation for further expansion and enhancement. In future versions, the functionality of the system can be extended by adding the following features:

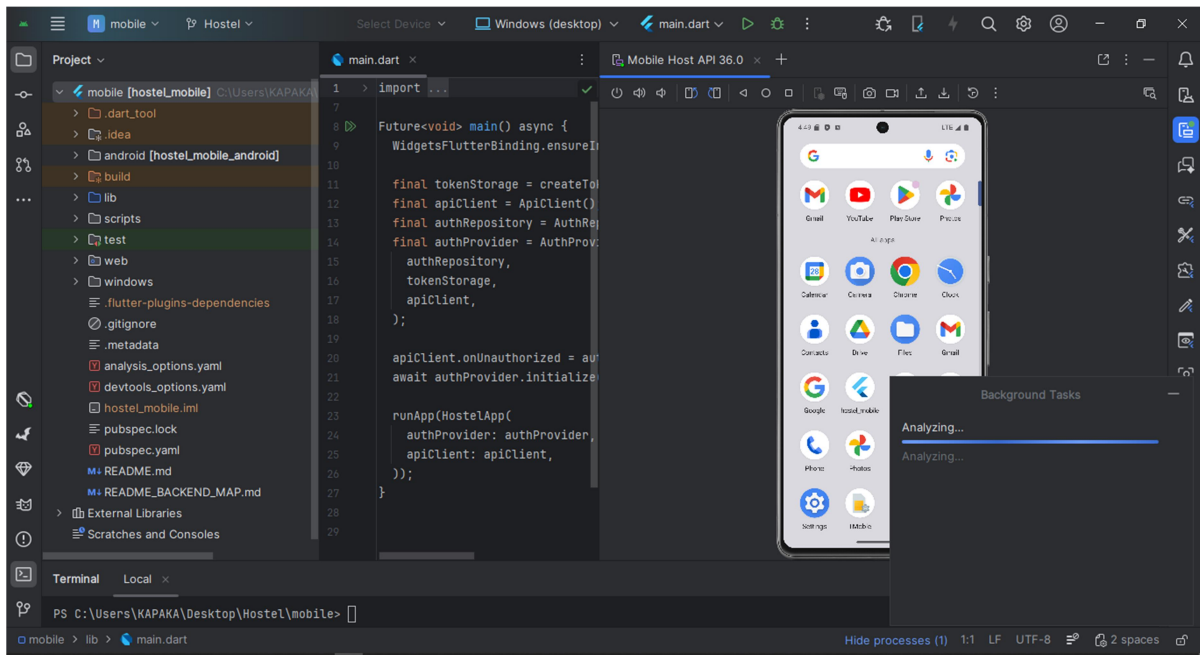
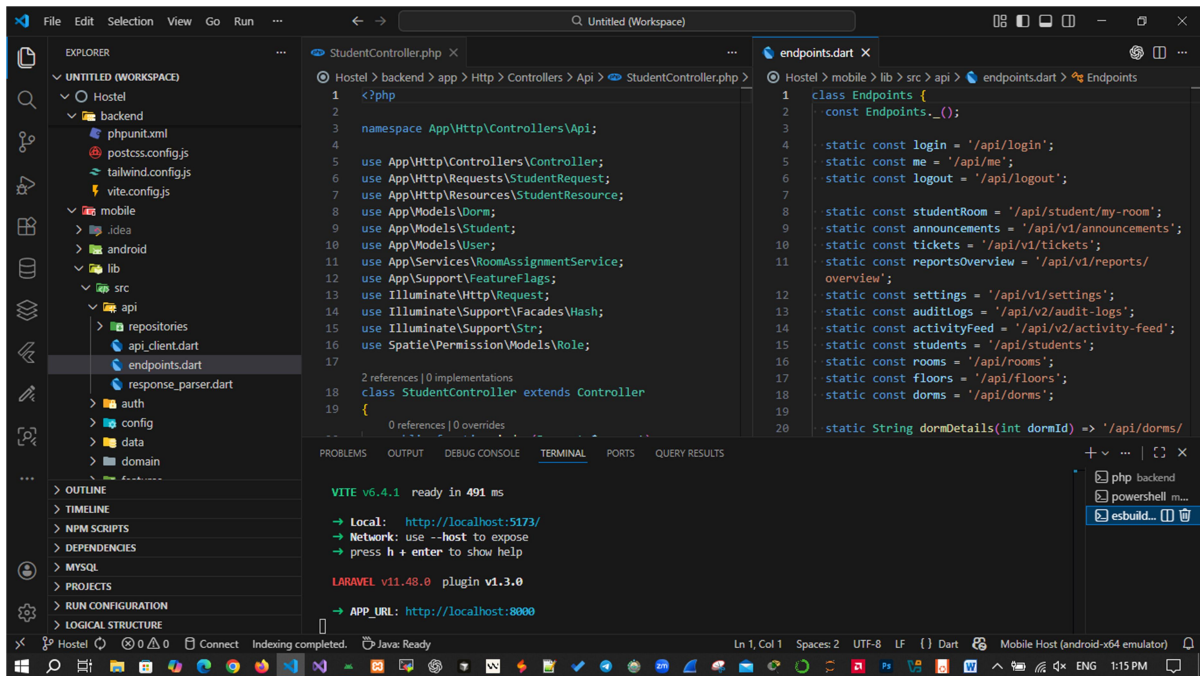
- Maintenance and service request management for dormitory rooms;
- Push notifications to inform students about important events and announcements;
- Generation of detailed reports in PDF and Excel formats;
- Payment and penalty management modules;
- Integration with existing university information systems;
- Advanced analytics and statistical dashboards for occupancy and usage trends.

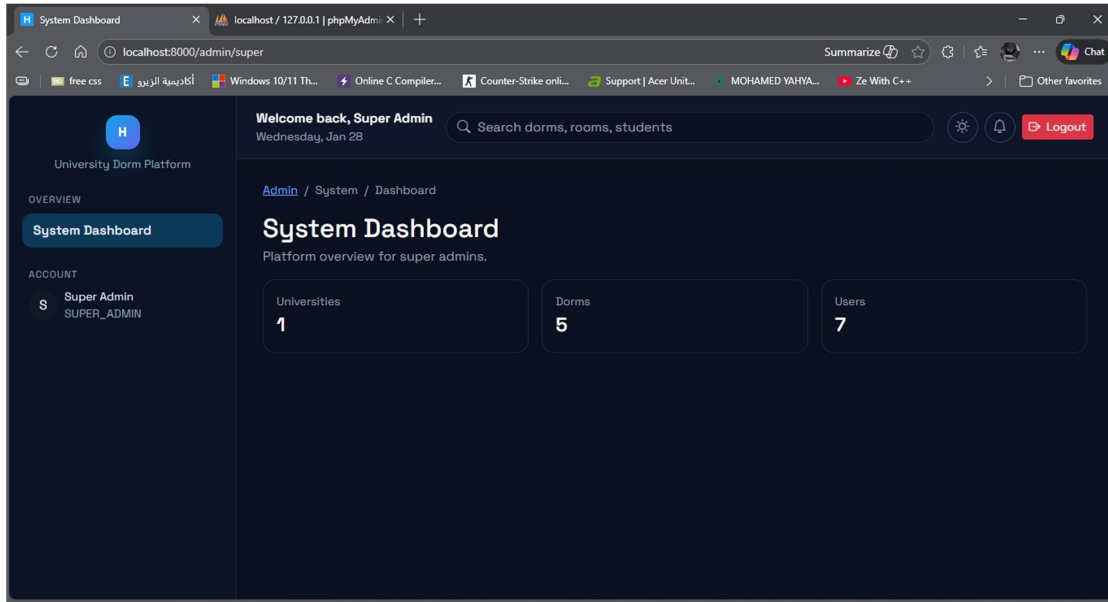
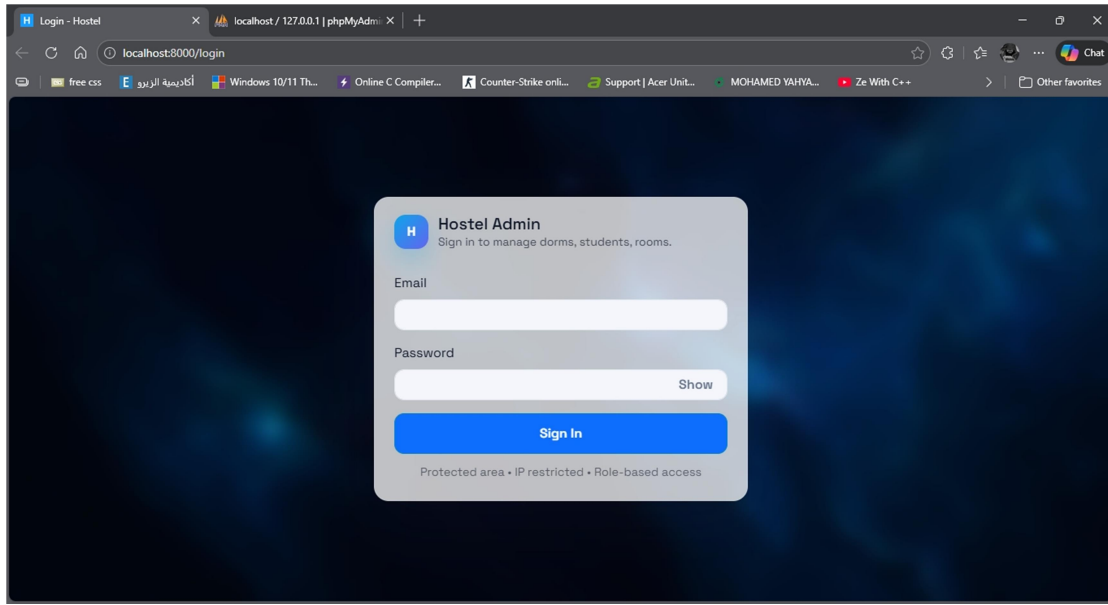
These improvements would increase the system's usability and allow it to be applied in a real university environment on a larger scale.

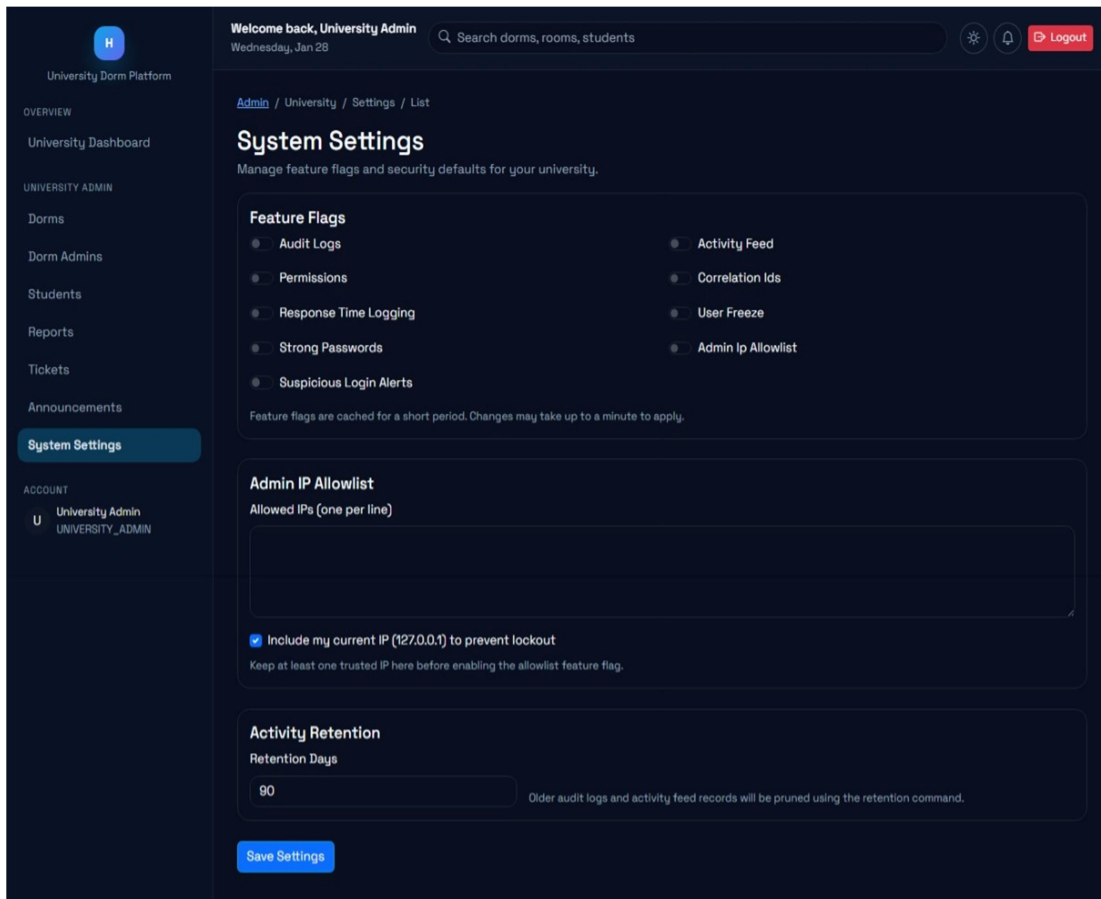
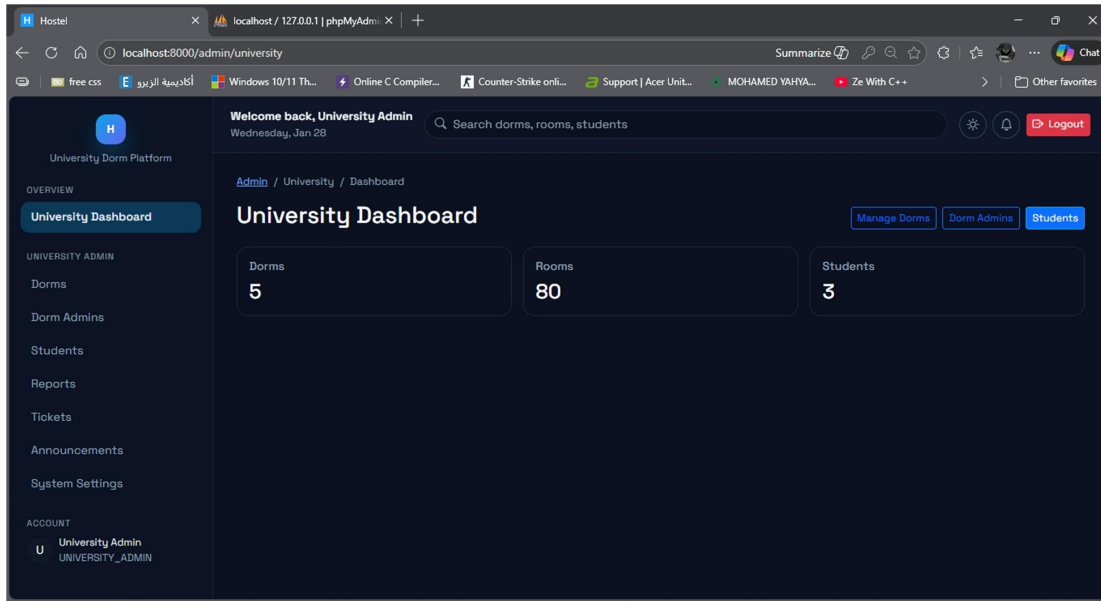
Conclusion

As a result of this course project, an online student dormitory management system was developed based on a Flutter mobile application and a Laravel backend with a MySQL database. The system implements core functionalities for managing dormitories, floors, rooms, and students, as well as a room assignment mechanism with capacity validation.

The system features a clear role-based access structure, ensuring secure and controlled access to data for different user categories. The chosen architecture and technologies provide reliability, scalability, and flexibility for future enhancements. Overall, the developed project demonstrates the effective application of modern mobile and web technologies to solve real-world administrative problems in the university environment.







H

University Dorm Platform

OVERVIEW

University Dashboard

UNIVERSITY ADMIN

Dorms

Dorm Admins

Students

Reports

Tickets

Announcements

System Settings

ACCOUNT

U

University Admin

UNIVERSITY_ADMIN

Welcome back, University Admin

Wednesday, Jan 28

Search dorms, rooms, students

Logout

Admin / University / Announcements / List

Announcements

Schedule and target dorm communications.

Search title or body

All statuses

All audiences

All dorms

Apply

Reset

Title	Audience	Status	Publish	Actions
Quiet hours reminder Dorm A	Dorm	Published	Jan 27, 2026 00:22	View Edit

H

University Dorm Platform

OVERVIEW

University Dashboard

UNIVERSITY ADMIN

Dorms

Dorm Admins

Students

Reports

Tickets

Announcements

System Settings

ACCOUNT

U

University Admin

UNIVERSITY_ADMIN

Welcome back, University Admin

Wednesday, Jan 28

Search dorms, rooms, students

Logout

Admin / University / Tickets / List

Tickets

Track maintenance issues across dorms.

Search subject, category, requester

All statuses

All priorities

All dorms

Apply

Reset

Export CSV

New Ticket

Ticket	Dorm	Priority	Status	Assignee	Actions
mo General	Dorm A	Medium	Open	Unassigned	View Edit
mo General	Dorm A	Medium	Open	Unassigned	View Edit
Air conditioner leak Maintenance	Dorm A	High	Open	Dorm Admin	View Edit

H

University Dorm Platform

OVERVIEW

University Dashboard

UNIVERSITY ADMIN

Dorms

Dorm Admins

Students

Reports

Tickets

Announcements

System Settings

ACCOUNT

U

University Admin

UNIVERSITY_ADMIN

Welcome back, University Admin

Wednesday, Jan 28

Q Search dorms, rooms, students

Logout

Admin / University / Reports / List

Reports

University occupancy and operational signals.

Dorms

5

Rooms

80

Students

3

Occupancy

1/315

Assignments Trend (14 days)

2026-01-15

2026-01-28

Tickets by Status (30 days)

Open

3

In progress

0

Resolved

0

Closed

0

Dorm Occupancy

Dorm	Capacity	Occupied	Percent
dorm 01	0	0	0%
Dorm A	315	1	0%
Dorm B	0	0	0%
Dorm C	0	0	0%
Dorm D	0	0	0%

H

University Dorm Platform

OVERVIEW

University Dashboard

UNIVERSITY ADMIN

Dorms

Dorm Admins

Students

Reports

Tickets

Announcements

System Settings

ACCOUNT

U

University Admin

UNIVERSITY_ADMIN

Welcome back, University Admin

Wednesday, Jan 28

Q Search dorms, rooms, students

Logout

Admin / University / Students / List

Students

University-wide overview and exports.

Import CSV

Export CSV

Search name, student no, email

All dorms

All statuses

Apply

Reset

Name	Student No	Email	Dorm	Status	Actions
Mohamed	102000	mhasanen114@gmail.com	Dorm A	ACTIVE	No actions
Omar	102030	omar@test.com	Dorm B	ACTIVE	No actions
Student One	2024001	student1@test.com	Dorm A	ACTIVE	No actions

H

University Dorm Platform

OVERVIEW

University Dashboard

UNIVERSITY ADMIN

Dorms

Dorm Admins

Students

Reports

Tickets

Announcements

System Settings

ACCOUNT

U

University Admin

UNIVERSITY_ADMIN

Welcome back, University Admin

Wednesday, Jan 28

Q Search dorms, rooms, students

Logout

Admin / University / Dorm Admins / List

Dorm Admins

Manage dorm managers and access levels.

Create Dorm Admin

Search by name, email, dorm

Search

Reset

Name	Email	Dorm	Status	Actions
Mohamed	kap@gmail.com	dorm 01	ACTIVE	Edit
Dorm Admin	dormadmin@test.com	Dorm A	ACTIVE	Edit

H

University Dorm Platform

OVERVIEW

University Dashboard

UNIVERSITY ADMIN

Dorms

Dorm Admins

Students

Reports

Tickets

Announcements

System Settings

ACCOUNT

U

University Admin

UNIVERSITY_ADMIN

Welcome back, University Admin

Wednesday, Jan 28

Q Search dorms, rooms, students

Logout

Admin / University / Dorms / List

Dorms

Manage inventory, capacity, and status.

Add Dorm

Search by name, code, address

All statuses

Sort: Name

Ascending

Apply

Reset

Code	Name	Address	Capacity	Rooms	Students	Status	Actions
dorm_01	dorm 01		0	0	0	ACTIVE	Edit Delete
DORM_A	Dorm A		80	80	2	ACTIVE	Edit Delete
DORM_B	Dorm B		80	0	1	ACTIVE	Edit Delete
DORM_C	Dorm C		80	0	0	ACTIVE	Edit Delete
DORM_D	Dorm D		80	0	0	ACTIVE	Edit Delete

H

University Dorm Platform

OVERVIEW

University Dashboard

UNIVERSITY ADMIN

Dorms

Dorm Admins

Students

Reports

Tickets

Announcements

System Settings

ACCOUNT

U

University Admin

UNIVERSITY_ADMIN

Welcome back, University Admin

Wednesday, Jan 28

Q Search dorms, rooms, students

Logout

Admin / University / Dashboard

University Dashboard

Manage Dorms

Dorm Admins

Students

Dorms

5

Rooms

80

Students

3

H

University Dorm Platform

OVERVIEW

Dorm Dashboard

DORM ADMIN

Floors

Rooms

Students

Assignments

Tickets

Announcements

ACCOUNT

D

Dorm Admin

DORM_ADMIN

Welcome back, Dorm Admin

Wednesday, Jan 28

Q Search dorms, rooms, students

Logout

Admin / Dorm / Announcements / List

Announcements

New Announcement

Share updates with dorm residents.

Search title or body

All statuses

Apply

Reset

Title	Status	Publish	Actions
Quiet hours reminder	Published	Jan 27, 2026 00:22	View Edit

H

University Dorm Platform

OVERVIEW

Dorm Dashboard

DORM ADMIN

Floors

Rooms

Students

Assignments

Tickets

Announcements

ACCOUNT

D

Dorm Admin

DORM_ADMIN

Welcome back, Dorm Admin

Wednesday, Jan 28

Q Search dorms, rooms, students

Logout

Admin / Dorm / Tickets / List

Tickets

Manage maintenance requests for your dorm.

Export CSV

New Ticket

Search subject, category, requester

All statuses

All priorities

Apply

Reset

Ticket	Priority	Status	Assignee	Actions
mo General	Medium	Open	Unassigned	View Edit
mo General	Medium	Open	Unassigned	View Edit
Air conditioner leak Maintenance	High	Open	Dorm Admin	View Edit

H

University Dorm Platform

OVERVIEW

Dorm Dashboard

DORM ADMIN

Floors

Rooms

Students

Assignments

Tickets

Announcements

ACCOUNT

D

Dorm Admin

DORM_ADMIN

Welcome back, Dorm Admin

Wednesday, Jan 28

Q Search dorms, rooms, students

Logout

Admin / Dorm / Assignments / List

Assignments

Track room allocation history and moves.

Search student or room

All statuses

All rooms

Apply

Reset

Student	Room	Assigned	Status	Actions
Student One 2024001	Room 101 Floor 1	Jan 27, 2026 Present	Active	Move Student

H

University Dorm Platform

OVERVIEW

Dorm Dashboard

DORM ADMIN

Floors

Rooms

Students

Assignments

Tickets

Announcements

ACCOUNT

Dorm Admin

DORM_ADMIN

Welcome back, Dorm Admin

Wednesday, Jan 28

Search dorms, rooms, students

Logout

Admin / Dorm / Students / List

Students

Track assignments, contacts, and status.

Export CSV

Add Student

Search by name, student no, email

All assignments

Active

Apply

Reset

Select all 0 selected

Bulk actions

Apply

☐	Student	Student No	Email	Room	Status	Actions
☒	Mohamed	102000	mhasanen114@gmail.com	Unassigned	Active	DetailsEditArchive
☐	Student One	2024001	student1@test.com	Room 101	Active	DetailsEditArchive

H

University Dorm Platform

OVERVIEW

Dorm Dashboard

DORM ADMIN

Floors

Rooms

Students

Assignments

Tickets

Announcements

ACCOUNT

D

Dorm Admin

DORM_ADMIN

Welcome back, Dorm Admin

Wednesday, Jan 28

Search dorms, rooms, students

Logout

Admin / Dorm / Rooms / List

Rooms

Visualize occupancy, capacity, and availability.

Export CSV

Add Room

Search by room number

All floors

All statuses

Availability

Active

Apply

Reset

Select all 0 selected

Bulk actions

Apply

Room 101

Floor 1

Occupancy

0 available beds

Details

Edit

Archive

FULL

1/1

Room 102

Floor 1

Occupancy

2 available beds

Details

Edit

Archive

AVAILABLE

0/2

Room 103

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 104

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 105

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 106

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 107

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 108

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 109

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 110

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 111

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 112

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 113

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 114

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 115

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 116

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 117

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 118

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 119

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 120

Floor 1

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 201

Floor 2

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 202

Floor 2

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 203

Floor 2

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Room 204

Floor 2

Occupancy

4 available beds

Details

Edit

Archive

AVAILABLE

0/4

Showing 1 to 24 of 80 results

<

1

2

3

4

>

H

University Dorm Platform

OVERVIEW

Dorm Dashboard

DORM ADMIN

Floors

Rooms

Students

Assignments

Tickets

Announcements

ACCOUNT

Dorm Admin

DORM_ADMIN

Welcome back, Dorm Admin

Wednesday, Jan 28

Search dorms, rooms, students

Logout

Admin / Dorm / Floors / List

Floors

Organize your dorm by floor and facility counts.

Add Floor

Search by number

ApplyReset

Select floors to delete in bulk.

Delete selected

☐	Number	Rooms	Bathrooms	Kitchens	Showers	Actions
☐	1	20	4	2	2	EditDelete
☐	2	20	4	2	2	EditDelete
☐	3	20	4	2	2	EditDelete
☐	4	20	4	2	2	EditDelete

H

University Dorm Platform

OVERVIEW

Dorm Dashboard

DORM ADMIN

Floors

Rooms

Students

Assignments

Tickets

Announcements

ACCOUNT

Dorm Admin

DORM_ADMIN

Welcome back, Dorm Admin

Wednesday, Jan 28

Search dorms, rooms, students

Logout

Admin / Dorm / Dashboard

Housing Dashboard

FloorsRoomsStudents

Floors

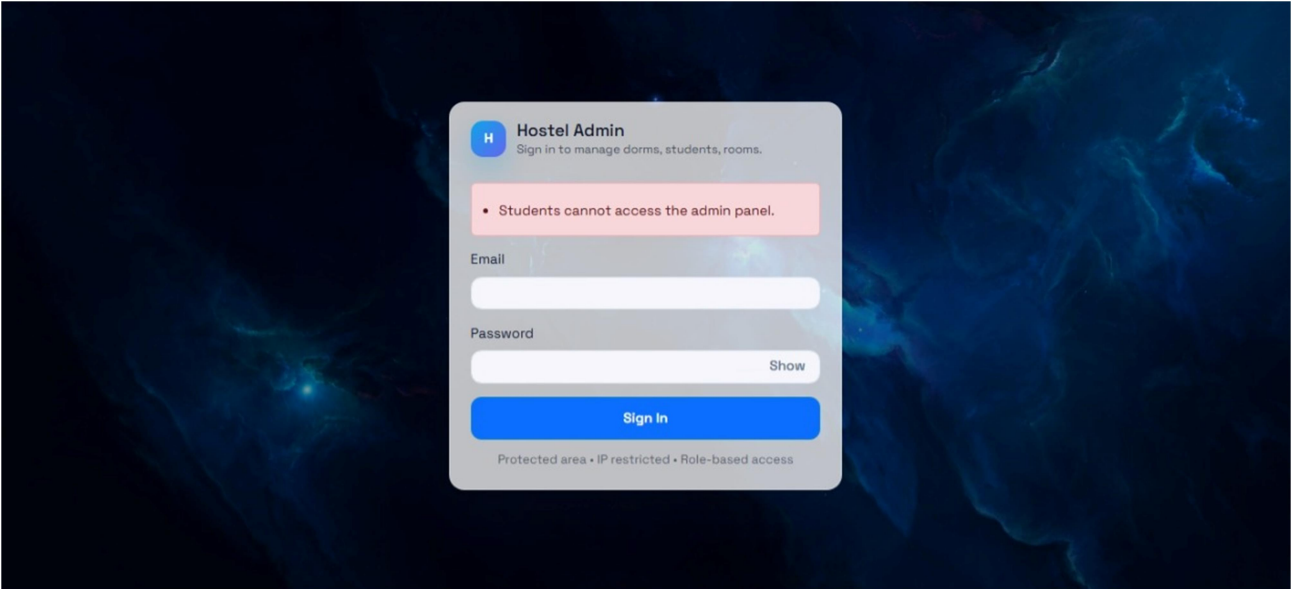
4

Rooms

80

Students

2



phpMyAdmin

Server: 127.0.0.1 > Database: hostel

Structure SQL Search Query Export Import Operations Privileges Routines Events Triggers Tracking

Recent Favorites

New

- 3tree_products_table
- a_khadma
- eg_lawyers
- hostel
 - New
 - admin_notifications
 - announcements
 - audit_logs
 - cache
 - cache_locks
 - dorms
 - dorm_admins
 - failed_jobs
 - floors
 - job_batches
 - migrations
 - model_has_permissions
 - model_has_roles
 - password_reset_tokens
 - permissions
 - personal_access_tokens
 - record_histories
 - roles
 - role_has_permissions
 - rooms
 - room_assignments
 - sessions
 - students
 - system_settings
 - tickets
 - ticket_comments
 - universities
 - users
 - user_freeze_logs

Type to filter these. Enter to see

Filters

Containing the word:

Table	Action	Rows	Type	Collation	Size	Overhead
☐ admin_notifications	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	32.0 K	-
☐ announcements	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	112.0 K	-
☐ audit_logs	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	80.0 K	-
☐ cache	Browse Structure Search Insert Empty Drop	18	InnoDB	utf8mb4_unicode_ci	16.0 K	-
☐ cache_locks	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K	-
☐ dorms	Browse Structure Search Insert Empty Drop	5	InnoDB	utf8mb4_unicode_ci	48.0 K	-
☐ dorm_admins	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_unicode_ci	48.0 K	-
☐ failed_jobs	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K	-
☐ floors	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	32.0 K	-
☐ jobs	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K	-
☐ job_batches	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K	-
☐ migrations	Browse Structure Search Insert Empty Drop	30	InnoDB	utf8mb4_unicode_ci	16.0 K	-
☐ model_has_permissions	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K	-
☐ model_has_roles	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	32.0 K	-
☐ password_reset_tokens	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K	-
☐ permissions	Browse Structure Search Insert Empty Drop	9	InnoDB	utf8mb4_unicode_ci	32.0 K	-
☐ personal_access_tokens	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	64.0 K	-
☐ record_histories	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	80.0 K	-
☐ roles	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	32.0 K	-
☐ role_has_permissions	Browse Structure Search Insert Empty Drop	14	InnoDB	utf8mb4_unicode_ci	32.0 K	-
☐ rooms	Browse Structure Search Insert Empty Drop	80	InnoDB	utf8mb4_unicode_ci	48.0 K	-
☐ room_assignments	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	64.0 K	-
☐ sessions	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_unicode_ci	48.0 K	-
☐ students	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	64.0 K	-
☐ system_settings	Browse Structure Search Insert Empty Drop	11	InnoDB	utf8mb4_unicode_ci	32.0 K	-
☐ tickets	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	96.0 K	-
☐ ticket_comments	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	64.0 K	-
☐ universities	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	32.0 K	-
☐ users	Browse Structure Search Insert Empty Drop	7	InnoDB	utf8mb4_unicode_ci	48.0 K	-
☐ user_freeze_logs	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	64.0 K	-
30 tables	Sum	199	InnoDB	utf8mb4_general_ci	1.3 MiB	0 B

☐ Check all With selected:

Print Data dictionary

Create new table

Table name: Number of columns: 4 Create

